

DevSpark
Miniblog
Api rest.
Desarrollado por Sebastián Pérez R.
Estudiante de Full Stack Development.
Henry.

Este proyecto es presentado para aprobar el M2.

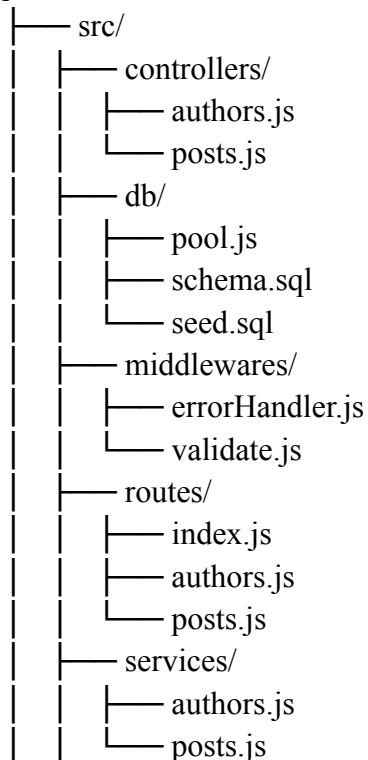
API REST para gestión de authors y posts, construida con Node.js, Express y PostgreSQL.
Proyecto integrador del Módulo 2 para DevSpark.

Tecnologías utilizadas

- **Node.js** — entorno de ejecución
- **Express** — framework web
- **PostgreSQL** — base de datos relacional
- **pg** — cliente PostgreSQL para Node.js
- **dotenv** — manejo de variables de entorno
- **Vitest** — framework de testing
- **Supertest** — testing de endpoints HTTP
- **Nodemon** — recarga automática en desarrollo

Estructura del proyecto

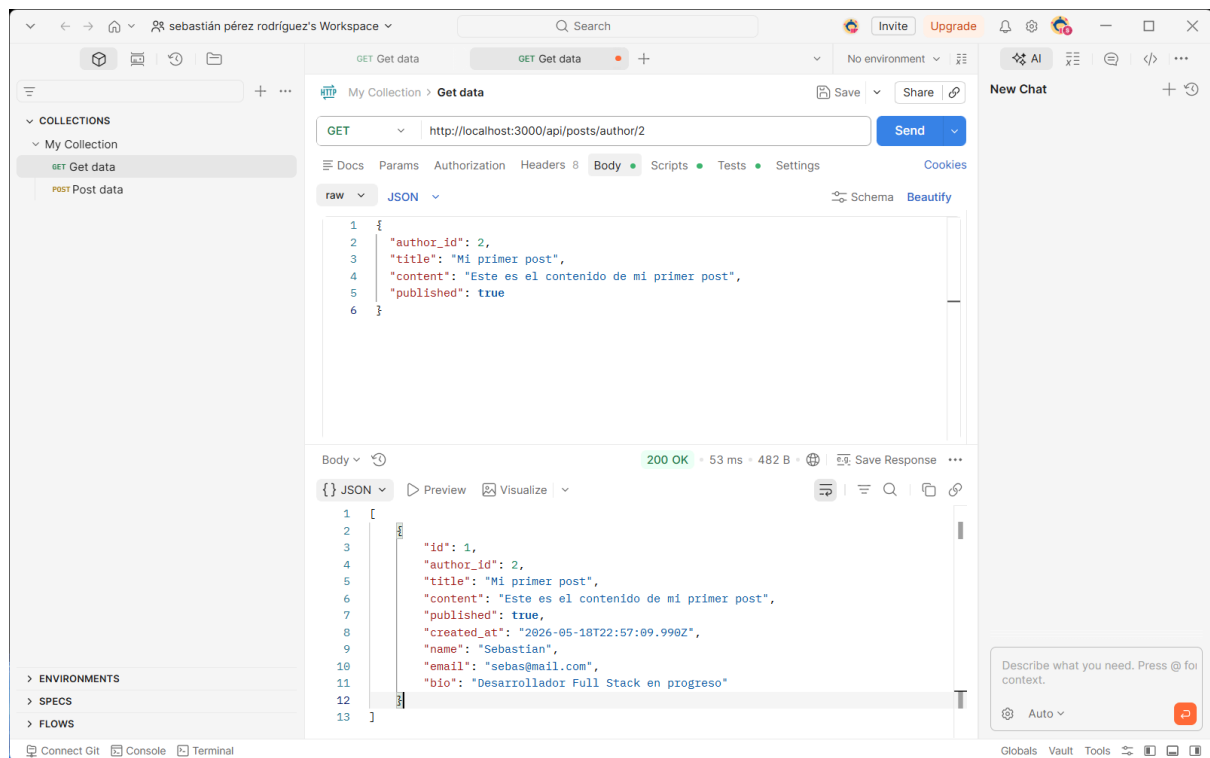
pi-m2/



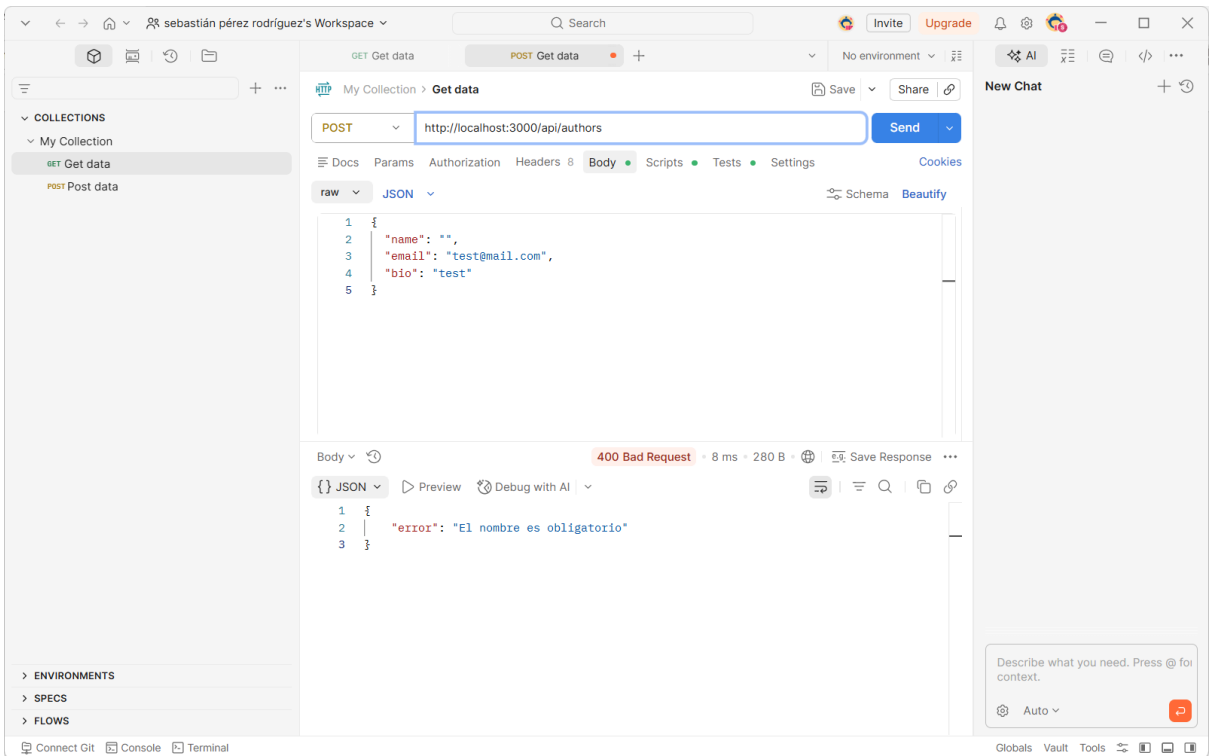
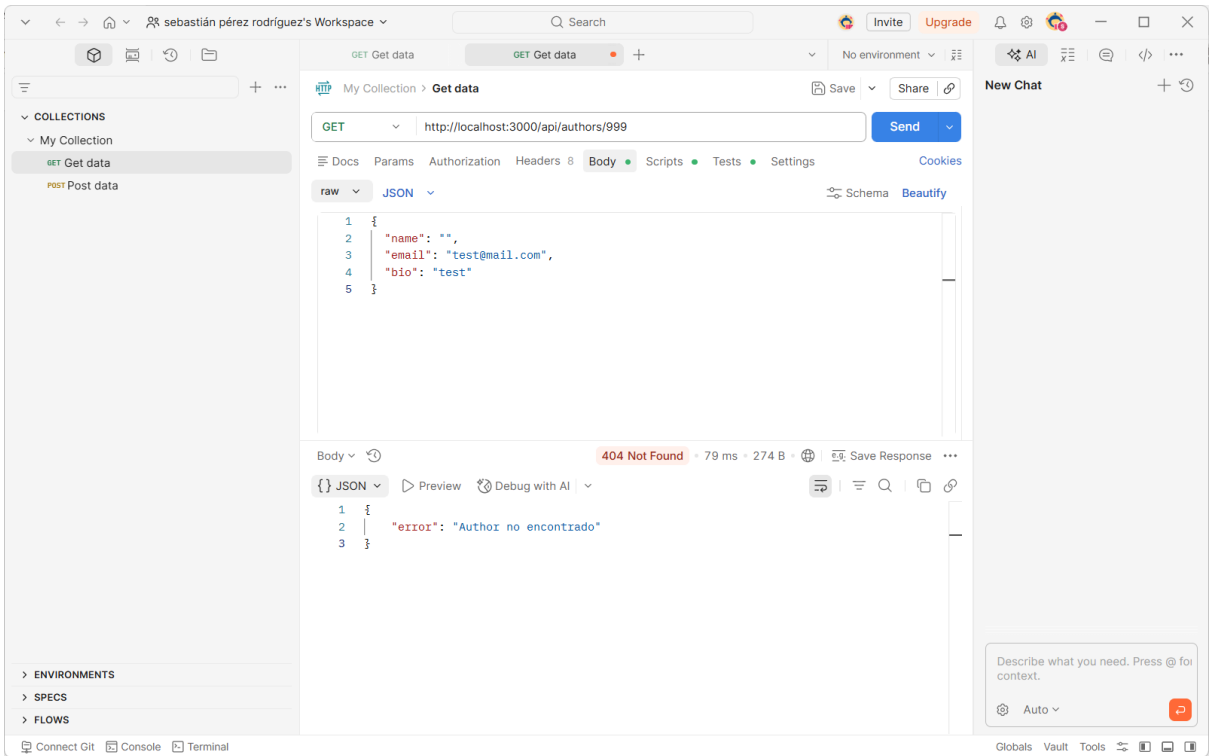
```
├── test/
│   ├── authors.test.js
│   └── posts.test.js
├── utils/
│   └── validators.js
├── app.js
├── server.js
├── .env.example
├── openapi.yaml
└── package.json
```

A continuación dejo capturas del desarrollo del proyecto y evidencias de los test, y diferentes pruebas realizadas.

Probando endpoints en Postman.



Probando flujo de errores.



Documentación en swagger.

The screenshot shows the Swagger Editor interface. On the left, the OpenAPI 3.0.0 definition is displayed in a code editor. The API is titled 'PI M2 - API REST' and is described as 'API REST para gestión de authors y posts'. The servers are configured with the URL 'http://localhost:3000/api' and the description 'Servidor local'. The paths are defined as follows:

- /authors:**
 - GET:** Listar todos los authors. Response: 200 (application/json, schema: array of Author).
 - POST:** Crear un author. Required: true. Response: 201 (Author creado), 400 (Datos inválidos).
- /authors/{id}:**
 - GET:** Obtener un author por id. Parameter: id (path).
 - PUT:** Actualizar un author.
 - DELETE:** Eliminar un author.
- /posts:**
 - GET:** Listar todos los posts.
 - POST:** Crear un post.
 - GET /posts/{id}:** Obtener un post por id.
 - PUT /posts/{id}:** Actualizar un post.
 - DELETE /posts/{id}:** Eliminar un post.

On the right, the 'default' server is selected, and a list of endpoints is shown with their respective HTTP methods and descriptions.

This screenshot shows the Swagger Editor interface with a detailed view of the DELETE endpoint for '/authors/{id}'. The left pane shows the OpenAPI definition, and the right pane provides a detailed view of the endpoint.

DELETE /authors/{id} Eliminar un author

Parameters:

Name	Description
id	required integer (path) id

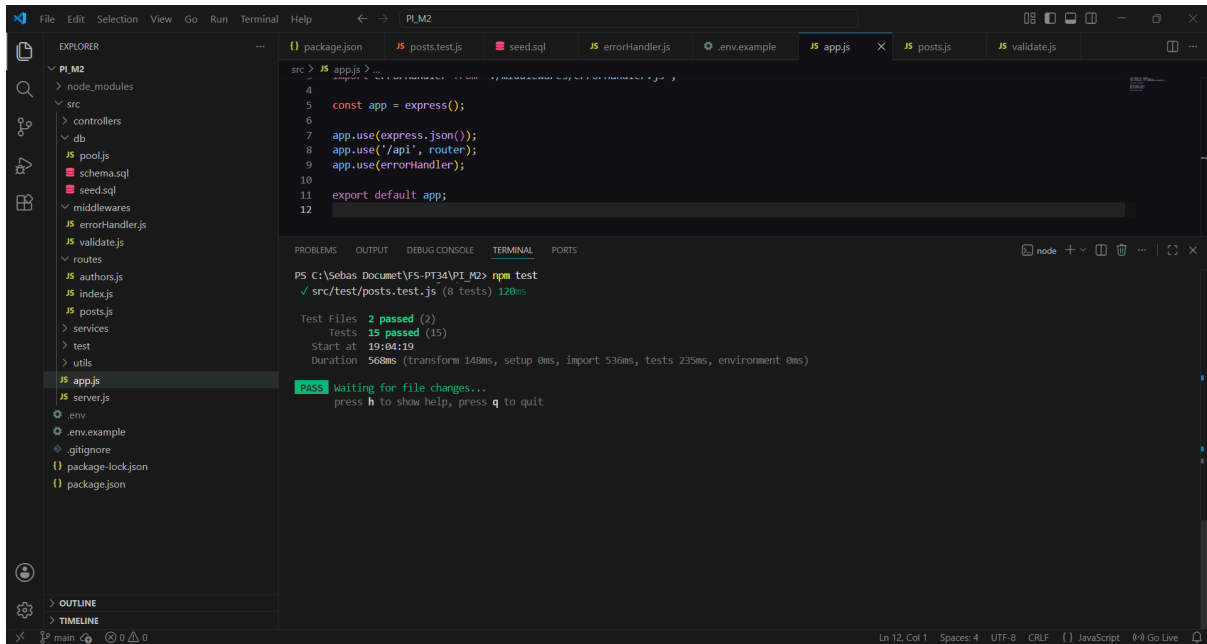
Responses:

Code	Description	Links
200	Author actualizado	No links
404	Author no encontrado	No links

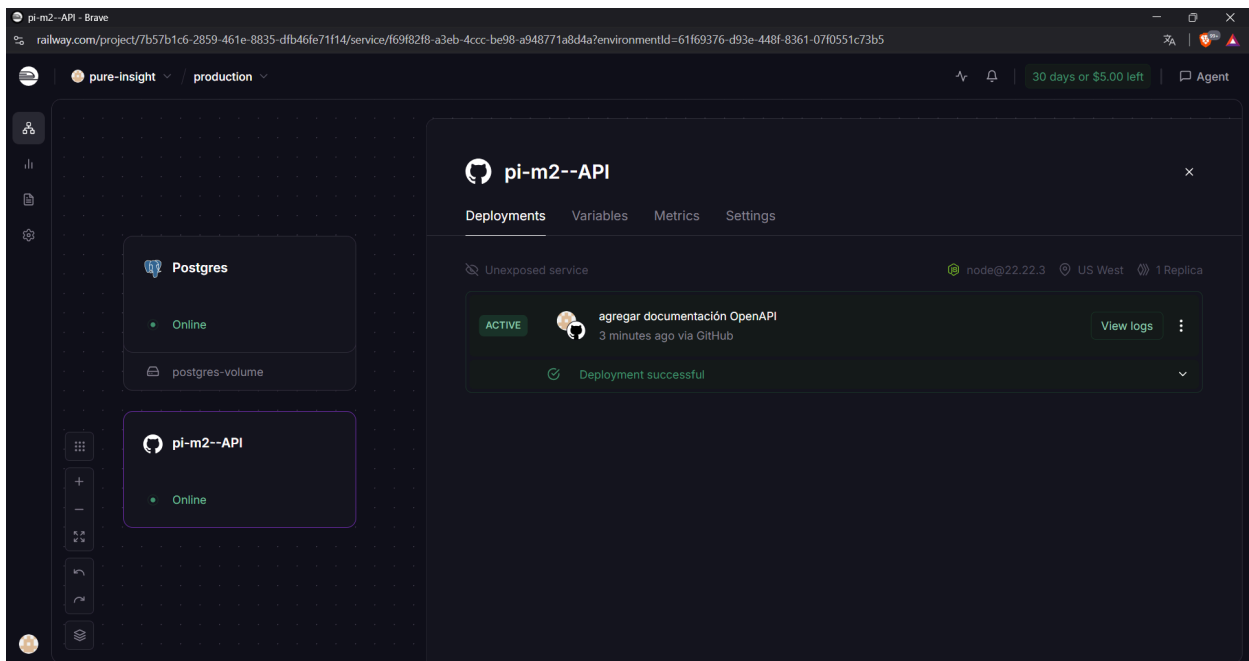
The detailed view of the 200 response shows the media type 'application/json' and an example of the response body:

```
{  "id": 1,  "name": "string",  "email": "string",  "bio": "string",  "created_at": "2025-05-13T06:13:38.556Z"}
```

Testing con Vitest y Supertest.



Deploy en Railway



Uso de la IA.

para el desarrollo de este proyecto decidí usar claude como apoyo en diferentes aspectos del proyecto, creando diferentes chats enfocados en estructuración, código y documentación.

